

Name: Kamraan Faiyaz Mulani

Roll no: 23101A0028

INFT A

Assignment 5 - Distributed Mutual Exclusion

Comparative Study: Ricart–Agrawala vs Token-Based Mutual Exclusion

Name: Kamraan, Assigned Real-Life System (Letter 'K'): Kubernetes

Why Kubernetes?

Since my name starts with 'K', I picked Kubernetes as the real-life distributed system for this assignment. Kubernetes is the container orchestration platform used by companies to manage clusters of machines running dozens or even thousands of containers at once. What makes it a good fit here is that Kubernetes constantly has to deal with the exact kind of problem this assignment is about: several nodes in the cluster (for example, controller replicas or kubelets updating shared cluster state in etcd) sometimes need exclusive access to the same resource - a lease, a lock on a leader-election key, or a piece of shared configuration - without stepping on each other. Kubernetes itself leans on a coordination service (etcd, which internally uses the Raft consensus protocol) for this, but the underlying problem it is solving is the same mutual exclusion problem that Ricart-Agrawala and token-based algorithms were originally designed to solve in general distributed systems. I have used Kubernetes as the running example throughout this answer to keep things grounded.

1. Ricart-Agrawala Algorithm

The Ricart-Agrawala algorithm is a permission-based (or 'non-token-based') approach to distributed mutual exclusion. The core idea is simple: before any process is allowed to enter the Critical Section (CS), it must first get explicit permission from every other process in the system. There is no central coordinator and no token moving around - a process earns its way into the CS purely through message exchange.

How it works

Imagine three nodes in a Kubernetes cluster's control plane, all of which briefly want to update the same shared configuration object at once. Here is the sequence Ricart-Agrawala follows:

- When a process (say, Node A) wants to enter the CS, it first increments its own Lamport logical clock and timestamps a REQUEST message with this value and its own process ID.
- It then broadcasts this REQUEST message to every other process in the system (Node B, Node C, and so on) - not just to a coordinator.
- When another process (Node B) receives the REQUEST, it checks its own state. If Node B is not interested in the CS, or if it is interested but its own request has a later timestamp (or the same timestamp but a larger process ID, which is the standard tie-breaking rule), then Node B immediately sends back a REPLY message.
- If Node B is currently inside the CS, it simply queues the incoming request and does not reply until it exits the CS.
- If Node B has also sent out its own REQUEST and its timestamp is smaller (i.e., it asked first), it delays its REPLY until it has finished with the CS.
- Node A is allowed to enter the CS only once it has collected REPLY messages from all other processes in the system.
- Once Node A finishes its work in the CS, it sends REPLYs to any requests it had been deferring, letting the next process go in.

The Lamport timestamp is what makes this fair and deadlock-free. Because every REQUEST carries a logical timestamp, and every process applies the same 'smaller timestamp wins, ties broken by process ID' rule, all the nodes end up agreeing - without ever talking to a central authority - on the same global ordering of requests. Whoever logically asked first gets in first. This matters a lot in something like Kubernetes' control plane, where two controller instances racing to patch the same object need a consistent way to decide who goes first, or the update history would become inconsistent.

The catch is cost: since every process has to message every other process (and get a reply back), Ricart-Agrawala needs $2(N-1)$ messages for every single CS entry, where N is the number of processes. That scales poorly as the cluster grows, but it does guarantee mutual exclusion, freedom from deadlock, and fairness (no starvation), which is why it is still a foundational algorithm to study even though production systems like Kubernetes use more optimized consensus-based mechanisms internally.

2. Token-Based Mutual Exclusion Algorithm

Token-based mutual exclusion takes a completely different approach. Instead of asking everyone for permission each time, the system maintains a single, unique token that circulates among the processes. Whoever is holding the token is the only process allowed to enter the Critical Section -

possession of the token is, by itself, the permission. There is no voting and no comparing of timestamps at the moment of entry.

How it works

- All N processes are logically arranged in a structure - commonly a ring, but it could also be organized around a coordinator that hands out the token on request.
- The token, which is really just a special message (in ring-based versions it often carries a queue of pending requests), is passed from one process to the next in a fixed sequence.
- A process that wants to enter the CS simply waits until the token physically arrives at its turn. If it does not need the CS when the token arrives, it just forwards the token onward to the next process without delay.
- If it does need the CS, it holds onto the token, performs its critical work, and only then passes the token on.
- Because only one copy of the token exists in the entire system, and only the token-holder may enter the CS, mutual exclusion is guaranteed automatically - two processes can never hold the token at the same time.

A good real-world parallel inside my chosen system, Kubernetes, is Kubernetes' leader election mechanism used by controller-manager and scheduler replicas. Only one replica is allowed to be the active 'leader' making decisions at any time, and it holds that role (much like holding a token, in spirit) via a lease object, renewing it periodically; if it stops renewing, the lease - the equivalent of the token - becomes available for another replica to acquire. It is not a literal ring-based token, but conceptually it mirrors the same idea: exclusive access is tied to possession of a single, well-defined 'permission object' rather than to a full round of voting.

Comparing the Two Approaches

Aspect	Ricart-Agrawala	Token-Based
Message overhead	High - needs $2(N-1)$ messages (REQUEST + REPLY) for every single CS entry.	Low and predictable - between 1 and N messages per entry, since the token is simply forwarded.
Handling simultaneous requests	Resolved cleanly using Lamport timestamps and the process-ID tie-breaker, so ordering is always consistent.	Resolved naturally by the fixed order in which the token travels; requests just wait their turn in the circulation/queue.
Fault tolerance	Fairly resilient - if one process crashes, others can often still	Weaker - if the token itself is lost (e.g., its holder crashes mid-CS), the whole system stalls until a costly

Aspect	Ricart-Agrawala	Token-Based
	proceed since there is no single point of failure carrying state.	token-regeneration or election protocol kicks in.

Conclusion

Both algorithms guarantee mutual exclusion, but they make very different trade-offs. Ricart-Agrawala is more chatty but does not depend on any single piece of state surviving - there is no 'token' that can get lost - and its use of Lamport timestamps gives it a clean, provably fair ordering. Token-based mutual exclusion is far cheaper in terms of messages once the system is running smoothly, but it introduces a single point of failure: if the token disappears, so does progress, unless the system has an explicit recovery mechanism. Seeing this through the lens of Kubernetes helped make the trade-off concrete for me - Kubernetes avoids both of these classic algorithms in their pure form and instead builds on Raft-based consensus (via etcd) precisely because it needs the fault tolerance of a permission-based scheme without paying the full $O(N)$ messaging cost of Ricart-Agrawala at massive cluster scale.